

Dynamic Programming

Dr. Kumkum Saxena

Contents

- Travelling salesman problem
- Optimal binary search tree (OBST)

Travelling salesman problem

Dynamic Programming

Traveling Salesman Problem

- Given a complete, weighted graph on n nodes, find the least weight Hamiltonian cycle, a cycle that visits every node once.
- Though this problem is easy enough to explain, it is very difficult to solve.
- Finding all the Hamiltonian cycles of a graph takes exponential time. Therefore, TSP is in the class NP.

If the TSP is so hard, why bother?

The TSP is interesting because it is in the class that is called NP-complete. If the TSP is found to have an algorithm that does not take exponential time, the other problems that are NP-complete will also be solved, and vice-a-versa.

If the TSP is so hard, why bother?

The TSP has many practical applications

- manufacturing
- plane routing
- telephone routing
- networks
- traveling salespeople
- structure of crystals

Problem Statement

- A traveller needs to visit all the cities from a list, where distances between all the cities are known and each city should be visited just once. What is the shortest possible route that he visits each city exactly once and returns to the origin city?

- Travelling salesman problem is the most notorious computational problem. We can use brute-force approach to evaluate every possible tour and select the best one. For n number of vertices in a graph, there are $(n - 1)!$ number of possibilities.
- Instead of brute-force using dynamic programming approach, the solution can be obtained in lesser time, though there is no polynomial time algorithm.
- Let us consider a graph $G = (V, E)$, where V is a set of cities and E is a set of weighted edges. An edge $e(u, v)$ represents that vertices u and v are connected. Distance between vertex u and v is $d(u, v)$, which should be non-negative.

- Suppose we have started at city **1** and after visiting some cities now we are in city **j**. Hence, this is a partial tour. We certainly need to know **j**, since this will determine which cities are most convenient to visit next. We also need to know all the cities visited so far, so that we don't repeat any of them. Hence, this is an appropriate sub-problem.
- For a subset of cities **S** $\in \{1, 2, 3, \dots, n\}$ that includes **1**, and **j** $\in$ **S**, let **C(S, j)** be the length of the shortest path visiting each node in **S** exactly once, starting at **1** and ending at **j**.

Consider the following fuzzy travelling salesman problem,

$$\text{Min. } \tilde{z} = \sum_{j=1}^n \tilde{c}_{ij} \tilde{x}_{ij}; [i= 1, 2, 3, \dots, n; j= 1, 2, 3, \dots, n]$$

Subject to the constraints:

$$(i) \sum_{i=1}^n \tilde{x}_{ij} = 1; j = 1, 2, \dots, n$$

$$(ii) \sum_{j=1}^n \tilde{x}_{ij} = 1; i = 1, 2, \dots, n.$$

Where $\tilde{x}_{ij} = \begin{cases} 1; & \text{if the salesman travel from city } i \text{ to city } j \\ 0; & \text{otherwise} \end{cases}$

$\tilde{c}_{ij}$ = Distance (or cost or time) of going from city i to city j .

$\tilde{z}$ = Min. The total cost of the matrix.

The Dynamic Programming proceeds as follows:-

Step-1 Consider the given travelling salesman problem in which he wants to find that route which has shortest distance.

Step-2 Consider set of 0 element, such that

$$g(2, \Phi) = c_{21}$$

$$g(3, \Phi) = c_{31}$$

$$g(4, \Phi) = c_{41}$$

Step-3 After completion of step-2, consider sets of 1 elements, such that

$$\begin{aligned}\text{Set } \{2\}: \quad & g(3, \{2\}) = c_{32} + g(2, \Phi) = c_{32} + c_{21} \\ & g(4, \{2\}) = c_{42} + g(2, \Phi) = c_{42} + c_{21}\end{aligned}$$

$$\begin{aligned}\text{Set } \{3\}: \quad & g(2, \{3\}) = c_{23} + g(3, \Phi) = c_{23} + c_{31} \\ & g(4, \{3\}) = c_{43} + g(3, \Phi) = c_{43} + c_{31}\end{aligned}$$

$$\begin{aligned}\text{Set } \{4\}: \quad & g(2, \{4\}) = c_{24} + g(4, \Phi) = c_{24} + c_{41} \\ & g(3, \{4\}) = c_{34} + g(4, \Phi) = c_{34} + c_{41}\end{aligned}$$

Step-4 After completion of step-3, consider sets of 2 elements, such that

$$\text{Set } \{2,3\}: \quad g(4, \{2,3\}) = \min \{c_{42} + g(2, \{3\}), c_{43} + g(3, \{2\})\}$$

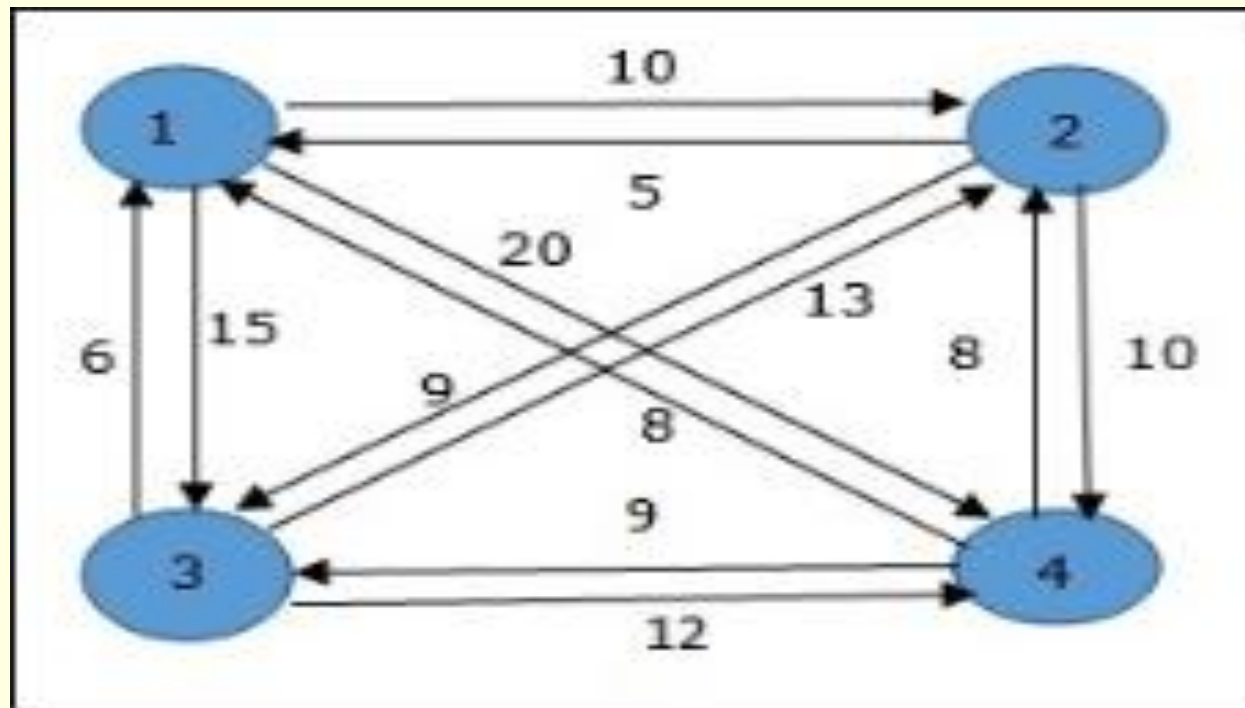
$$\text{Set } \{2,4\}: \quad g(3, \{2,4\}) = \min \{c_{32} + g(2, \{4\}), c_{34} + g(4, \{2\})\}$$

$$\text{Set } \{3,4\}: \quad g(2, \{3,4\}) = \min \{c_{23} + g(3, \{4\}), c_{24} + g(4, \{3\})\}$$

Step-5 After completion of step-4, Find the length of an optimal tour:

$$f = g(1, \{2,3,4\}) = \min \{c_{12} + g(2, \{3,4\}), c_{13} + g(3, \{2,4\}), c_{14} + g(4, \{2,3\})\}$$

Step-6 After completion of step-5, Find the Optimal TSP tour.



From the above graph, the following table is prepared.

	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Example 1. A salesman must visit from city to city to maintain his accounts. He has to leave his home city A and visit other cities once and return home. The cost of going from city to city is shown in the table. Find the least cost route

Distance matrix

$$\begin{pmatrix} 0 & 2 & 9 & 10 \\ 1 & 0 & 6 & 4 \\ 15 & 7 & 0 & 8 \\ 6 & 3 & 12 & 0 \end{pmatrix}$$

Solution:

$$g(2, \Phi) =$$

$$c_{21} = 1$$

$$g(3, \Phi) = c_{31}$$

$$= 15$$

$$g(4, \Phi) =$$

$$c_{41} = 6$$

$k = 1$, consider sets of 1 element:

Set {2}: $g(3, \{2\}) = c_{32} + g(2, \Phi) = c_{32} + c_{21} = 7 + 1 = 8$

$$g(4, \{2\}) = c_{42} + g(2, \Phi) = c_{42} + c_{21} = 3 + 1 = 4$$

Set {3}: $g(2, \{3\}) = c_{23} + g(3, \Phi) = c_{23} + c_{31} = 6 + 15 = 21$

$$g(4, \{3\}) = c_{43} + g(3, \Phi) = c_{43} + c_{31} = 12 + 15 = 27$$

Set {4}: $g(2, \{4\}) = c_{24} + g(4, \Phi) = c_{24} + c_{41} = 4 + 6 = 10$

$$g(3, \{4\}) = c_{34} + g(4, \Phi) = c_{34} + c_{41} = 8 + 6 = 14$$

k = 2, consider sets of 2 elements:

$$\text{Set } \{2,3\}: g(4, \{2,3\}) = \min \{c_{42} + g(2, \{3\}), c_{43} + g(3, \{2\})\} = \min \{3+21, 12+8\} = \min \{24, 20\} = 20$$

$$\text{Set } \{2,4\}: g(3, \{2,4\}) = \min \{c_{32} + g(2, \{4\}), c_{34} + g(4, \{2\})\} = \min \{7+10, 8+4\} = \min \{17, 12\} = 12$$

$$\text{Set } \{3,4\}: g(2, \{3,4\}) = \min \{c_{23} + g(3, \{4\}), c_{24} + g(4, \{3\})\} = \min \{6+14, 4+27\} = \min \{20, 31\} = 20$$

Length of an optimal tour:

$$\begin{aligned} f = g(1, \{2,3,4\}) &= \min \{ c_{12} + g(2, \{3,4\}), c_{13} + g(3, \{2,4\}), c_{14} + g(4, \{2,3\}) \} \\ &= \min \{ 2 + 20, 9 + 12, 10 + 20 \} = \min \{ 22, 21, 30 \} = 21 \end{aligned}$$

Successor of node 1: $g(2, \Phi) = c_{21} = 1$

Successor of node 2: $g(4, \{2\}) = 4$

Successor of node 4: $g(3, \{2, 4\}) = 12$

Successor of node 3: $g(1, \{2, 3, 4\}) = 21$

Optimal TSP tour: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$

Travelling Salesman Problem

for each subset S of the cities with
 $|S| \geq 2$ and each $u, v \in S$

$K[S, u, v]$ the length of the shortest path that

- * starts at u
- * ends at v
- * visits all cities in S

How large is K ?

Travelling Salesman Problem

for each subset S of the cities with
 $|S| \geq 2$ and each $u, v \in S$

$K[S, u, v]$ the length of the shortest path that

- * starts at u
- * ends at v
- * visits all cities in S

How large is K ?

$$\approx 2^n n^2$$

Travelling Salesman Problem

$K[S, u, v]$

some vertex $w \in S - \{u, v\}$
must be visited first

$d(u, w)$ = we get to w

$K[S - u, w, v]$ = we need to get from w to v
and visit all vertices in $S - u$

Travelling Salesman Problem

$K[S,u,v]$ the length of the shortest path that

- * starts at u
- * ends at v
- * visits all cities in S

if $S=\{u,v\}$ then $K[S,u,v]=d(u,v)$

if $|S|>2$ then

$$K[S,u,v] = \min_w K[S-\{u,w\},v] + d(u,w)$$

$$w \in S - \{u,v\}$$

Travelling Salesman Problem

if $S=\{u,v\}$ then $K[S,u,v]=d(u,v)$

if $|S|>2$ then

$K[S,u,v] = \min_{w \in S - \{u,v\}} K[S - u, w, v] + d(u, w)$

$w \in S - \{u, v\}$

Running time = ?

$$K \approx 2^n n^2$$

Travelling Salesman Problem

if $S=\{u,v\}$ then $K[S,u,v]=d(u,v)$

if $|S|>2$ then

$K[S,u,v] = \min_{w \in S - \{u,v\}} K[S - u, w, v] + d(u, w)$

$w \in S - \{u, v\}$

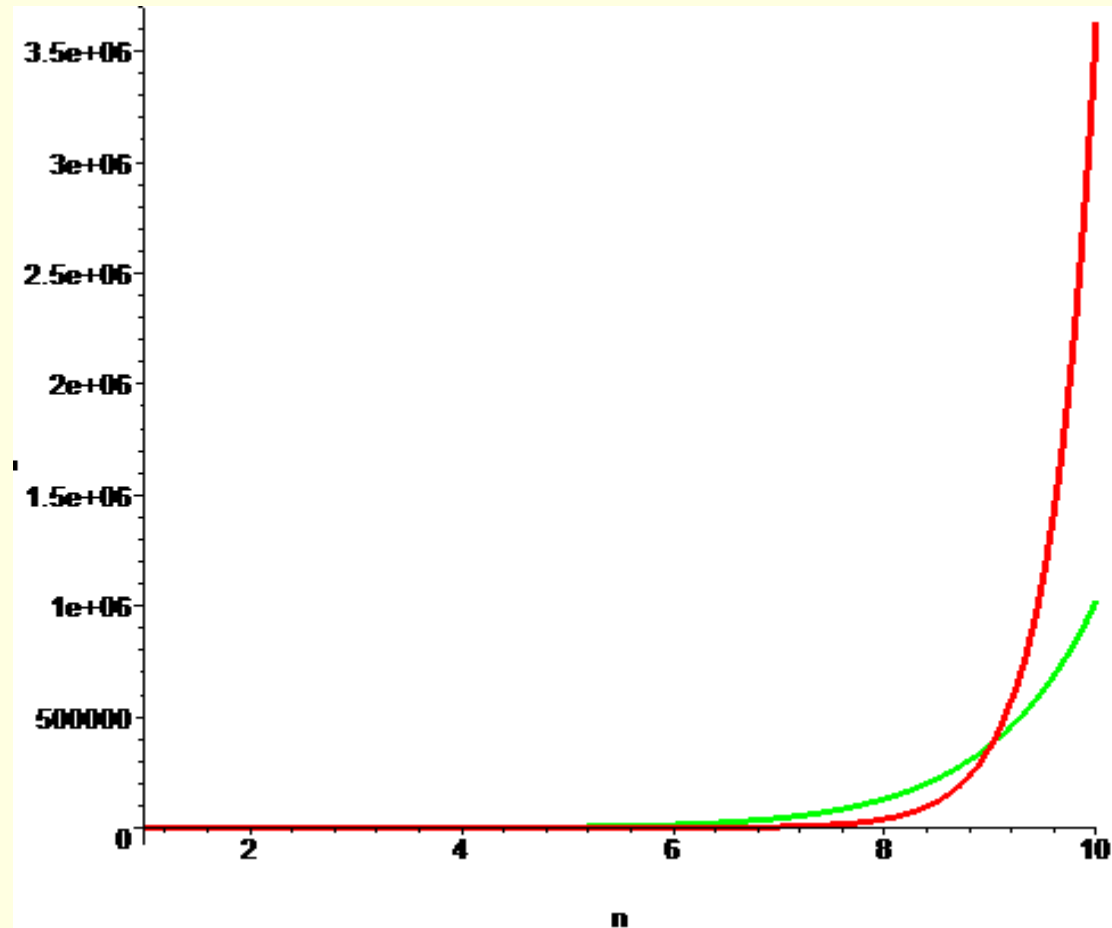
Running time = $O(n^3 2^n)$

$$K \approx 2^n n^2$$

Travelling Salesman Problem

dynamic programming = $O(n^3 2^n)$

brute force = $O(n!)$



Analysis

- There are at the most $(2^n \cdot n)$ sub-problems and each one takes linear time to solve. Therefore, the total running time is $O(2^n \cdot n^2)$.

Optimal Binary Search Tree(OBST)

Dynamic Programming

Optimal Binary Search Trees

■ Problem

- Given sequence $K = k_1 < k_2 < \dots < k_n$ of n sorted keys,
with a search probability p_i for each key k_i .
- Want to build a binary search tree (BST)
with minimum expected search cost.
- Actual cost = # of items examined.
- For key k_i , cost = $\text{depth}_T(k_i) + 1$, where $\text{depth}_T(k_i)$ = depth of k_i in BST T .

Expected Search Cost

$E[\text{search cost in } T]$

$$= \sum_{i=1}^n (\text{depth}_T(k_i) + 1) \cdot p_i$$

$$= \sum_{i=1}^n \text{depth}_T(k_i) \cdot p_i + \sum_{i=1}^n p_i$$

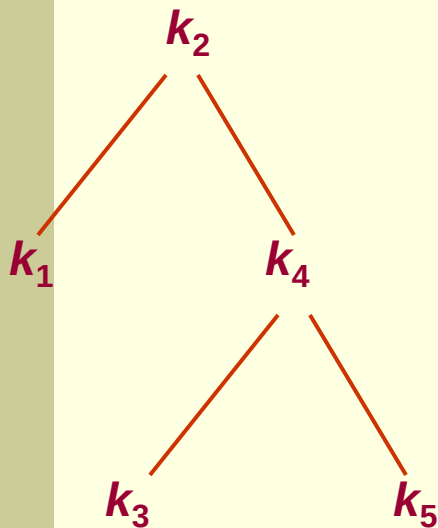
$$= 1 + \sum_{i=1}^n \text{depth}_T(k_i) \cdot p_i \quad (15.16)$$

Sum of probabilities is 1.

Example

- Consider 5 keys with these search probabilities:

$$p_1 = 0.25, p_2 = 0.2, p_3 = 0.05, p_4 = 0.2, p_5 = 0.3.$$



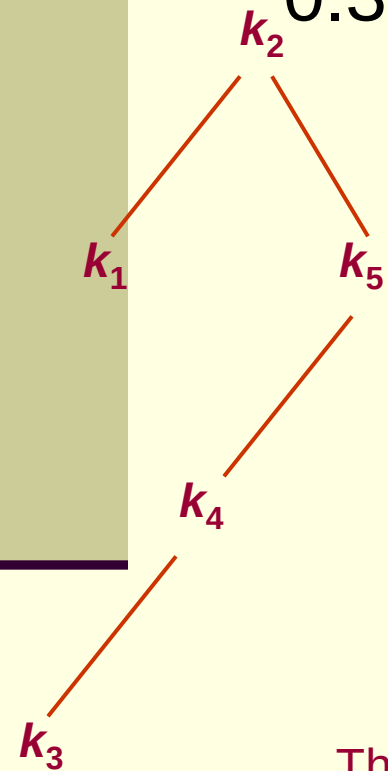
i	$\text{depth}_T(k_i)$	$\text{depth}_T(k_i) \cdot p_i$
1	1	0.25
2	0	0
3	2	0.1
4	1	0.2
5	2	0.6
		1.15

Therefore, $E[\text{search cost}] = 2.15$.

Example

- $p_1 = 0.25, p_2 = 0.2, p_3 = 0.05, p_4 = 0.2, p_5 = 0.3$.

i	$\text{depth}_T(k_i)$	$\text{depth}_T(k_i) \cdot p_i$
1	1	0.25
2	0	0
3	3	0.15
4	2	0.4
5	1	0.3
		1.10



Therefore, $E[\text{search cost}] = 2.10$.

This tree turns out to be optimal for this set of keys.

Example

■ Observations:

- Optimal BST **may not** have smallest height.
- Optimal BST **may not** have highest-probability key at root.
- Build by exhaustive checking?
 - Construct each n -node BST.
 - For each,
assign keys and compute expected search cost.
 - But there are $\Omega(4^n/n^{3/2})$ different BSTs with n nodes.

- A Binary Search Tree (BST) is a tree where the key values are stored in the internal nodes. The external nodes are null nodes. The keys are ordered lexicographically, i.e. for each internal node all the keys in the left sub-tree are less than the keys in the node, and all the keys in the right sub-tree are greater.
- When we know the frequency of searching each one of the keys, it is quite easy to compute the expected cost of accessing each node in the tree. An optimal binary search tree is a BST, which has minimal expected cost of locating each node

- Search time of an element in a BST is $O(n)$, whereas in a Balanced-BST search time is $O(\log n)$. Again the search time can be improved in Optimal Cost Binary Search Tree, placing the most frequently used data in the root and closer to the root element, while placing the least frequently used data near leaves and in leaves.
- Here, the Optimal Binary Search Tree Algorithm is presented. First, we build a BST from a set of provided n number of distinct keys $\langle k_1, k_2, k_3, \dots, k_n \rangle$. Here we assume, the probability of accessing a key K_i is p_i . Some dummy keys ($d_0, d_1, d_2, \dots, d_n$) are added as some searches may be performed for the values which are not present in the Key set K . We assume, for each dummy key d_i probability of access is q_i .

```

Optimal-Binary-Search-Tree(p, q, n)
e[1..n + 1, 0..n],
w[1..n + 1, 0..n],
root[1..n + 1, 0..n]
for i = 1 to n + 1 do
    e[i, i - 1] := qi - 1
    w[i, i - 1] := qi - 1
for l = 1 to n do
    for i = 1 to n - l + 1 do
        j = i + l - 1
        e[i, j] := ∞
        w[i, i] := w[i, i - 1] + pj + qj
        for r = i to j do
            t := e[i, r - 1] + e[r + 1, j] + w[i, j]
            if t < e[i, j]
                e[i, j] := t
                root[i, j] := r
return e and root

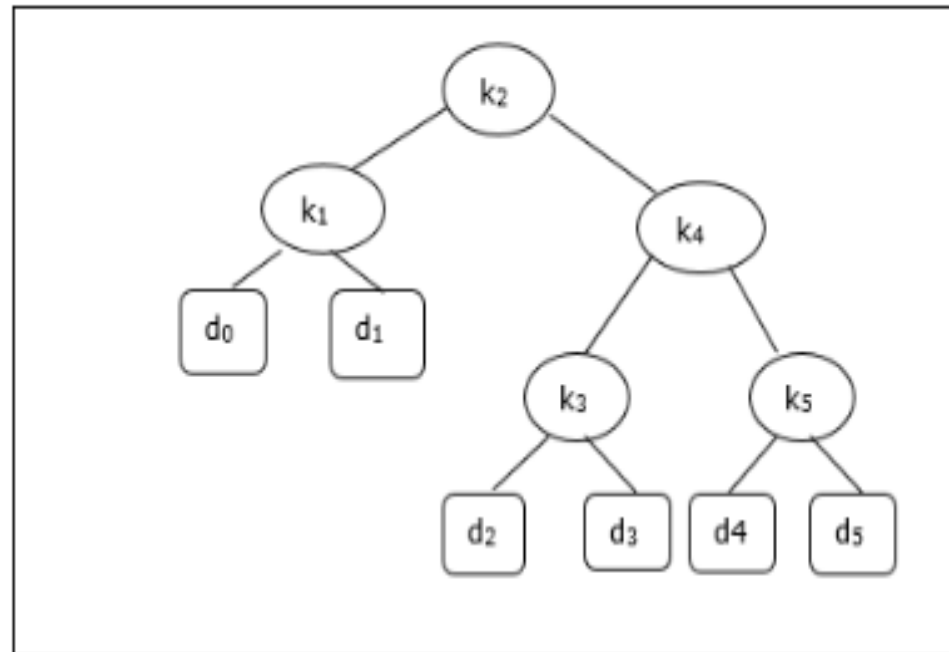
```

Analysis

- The algorithm requires $O(n^3)$ time, since three nested **for** loops are used. Each of these loops takes on at most n values.

Example

Considering the following tree, the cost is 2.80, though this is not an optimal result.



Node	Depth	Probability	Contribution
k ₁	1	0.15	0.30
k ₂	0	0.10	0.10
k ₃	2	0.05	0.15
k ₄	1	0.10	0.20
k ₅	2	0.20	0.60
d ₀	2	0.05	0.15
d ₁	2	0.10	0.30
d ₂	3	0.05	0.20
d ₃	3	0.05	0.20
d ₄	3	0.05	0.20
d ₅	3	0.10	0.40
Total			2.80

In the following tables, column index is **i** and row index is **j**.

e	1	2	3	4	5	6
5	2.75	2.00	1.30	0.90	0.50	0.10
4	1.75	1.20	0.60	0.30	0.05	
3	1.25	0.70	0.25	0.05		
2	0.90	0.40	0.05			
1	0.45	0.10				
0	0.05					

w	1	2	3	4	5	6
5	1.00	0.80	0.60	0.50	0.35	0.10
4	0.70	0.50	0.30	0.20	0.05	
3	0.55	0.35	0.15	0.05		
2	0.45	0.25	0.05			
1	0.30	0.10				
0	0.05					

root	1	2	3	4	5
5	2	4	5	5	5
4	2	2	4	4	
3	2	2	3		
2	1	2			
1	1				

1.Preface

- OBST is one special kind of advanced tree.
- It focus on how to reduce the cost of the search of the BST.
- It may not have the lowest height !
- It needs 3 tables to record probabilities, cost, and root.

2.Premise

- It has n keys (representation $k_1, k_2, \dots, k_n$) in sorted order (so that $k_1 < k_2 < \dots < k_n$), and we wish to build a binary search tree from these keys. For each k_i , we have a probability p_i that a search will be for k_i .
 - In contrast of, some searches may be for values not in k_i , and so we also have $n+1$ “dummy keys” $d_0, d_1, \dots, d_n$ representing not in k_i .
 - In particular, d_0 represents all values less than k_1 , and d_n represents all values greater than k_n , and for $i=1, 2, \dots, n-1$, the dummy key d_i represents all values between k_i and k_{i+1} .
- * The dummy keys are leaves (external nodes), and the data keys mean internal nodes.**

3. Formula & Prove

- The case of search are two situations, one is success, and the other, without saying, is failure.

- We can get the first statement :

$$(i=1\sim n) \sum p_i + (i=0\sim n) \sum q_i = 1$$

↑
Success

↑
Failure

- Because we have probabilities of searches for each key and each dummy key, we can determine the expected cost of a search in a given binary search tree T . Let us assume that the actual cost of a search is the number of nodes examined, i.e., the depth of the node found by the search in T , plus 1. Then the expected cost of a search in T is : (The second statement)
- $E[\text{search cost in } T]$

$$= \sum_{i=1}^n p_i \cdot (\text{depth}_T(k_i) + 1) + \sum_{i=0}^n q_i \cdot (\text{depth}_T(d_i) + 1)$$

$$= 1 + \sum_{i=1}^n p_i \cdot \text{depth}_T(k_i) + \sum_{i=0}^n q_i \cdot \text{depth}_T(d_i)$$

Where depth_T denotes a node's depth in the tree T .

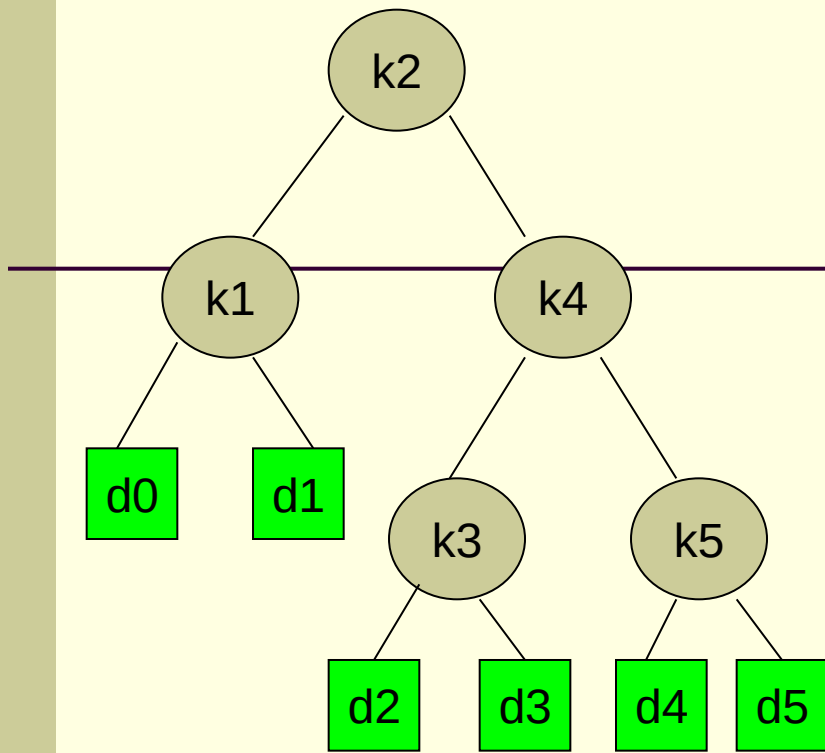


Figure (a)

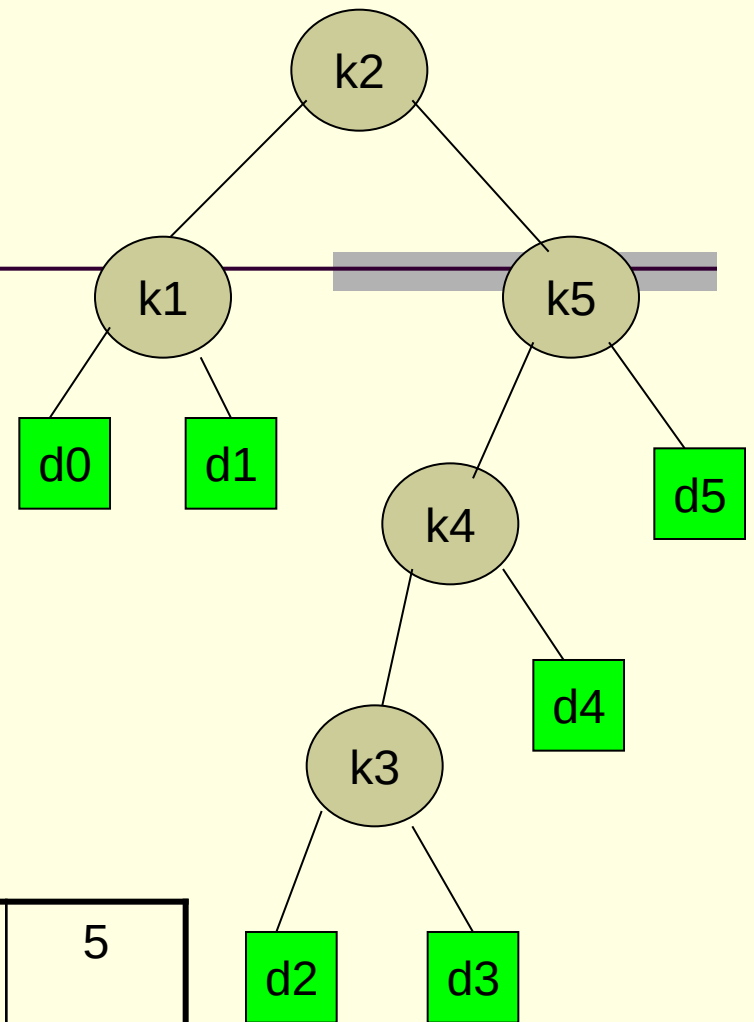


Figure (b)

i	0	1	2	3	4	5
pi		0.15	0.10	0.05	0.10	0.20
qi	0.05	0.10	0.05	0.05	0.05	0.10

By Figure (a), we can calculate the expected search cost node by node:

Node#	Depth	probability	cost
k1	1	0.15	0.30
k2	0	0.10	0.10
k3	2	0.05	0.15
k4	1	0.10	0.20
K5	2	0.20	0.60
d0	2	0.05	0.15
d1	3	0.10	0.30
d2	3	0.05	0.20
d3	3	0.05	0.20
d4	3	0.05	0.20
d5	3	0.10	0.40

Cost=
Probability *
(Depth+1)

- And the total cost = $(0.30 + 0.10 + 0.15 + 0.20 + 0.60 + 0.15 + 0.30 + 0.20 + 0.20 + 0.20 + 0.40)$
 $= 2.80$
- So Figure (a) costs 2.80 ,on another, the Figure (b) costs 2.75, and that tree is really optimal.
- We can see the height of (b) is more than (a) , and the key k5 has the greatest search probability of any key, yet the root of the OBST shown is k2.(The lowest expected cost of any BST with k5 at the root is 2.85)

Step1:The structure of an OBST

- To characterize the optimal substructure of OBST, we start with an observation about subtrees. Consider any subtree of a BST. It must contain keys in a contiguous range $k_i, \dots, k_j$, for some $1 \leq i \leq j \leq n$. In addition, a subtree that contains keys $k_i, \dots, k_j$ must also have as its leaves the dummy keys $d_{i-1}, \dots, d_j$.

- We need to use the optimal substructure to show that we can construct an optimal solution to the problem from optimal solutions to subproblems. Given keys $k_i, \dots, k_j$, one of these keys, say k_r ($1 \leq r \leq j$), will be the root of an optimal subtree containing these keys. The left subtree of the root k_r will contain the keys $(k_i, \dots, k_{r-1})$ and the dummy keys $(d_{i-1}, \dots, d_{r-1})$, and the right subtree will contain the keys $(k_{r+1}, \dots, k_j)$ and the dummy keys $(d_r, \dots, d_j)$. As long as we examine all candidate roots k_r , where $1 \leq r \leq j$, and we determine all optimal binary search trees containing $k_i, \dots, k_{r-1}$ and those containing $k_{r+1}, \dots, k_j$, we are guaranteed that we will find an OBST.

- There is one detail worth nothing about “empty” subtrees. Suppose that in a subtree with keys $k_i, \dots, k_j$, we select k_i as the root. By the above argument, k_i ’s left subtree contains the keys $k_i, \dots, k_{i-1}$. It is natural to interpret this sequence as containing no keys. It is easy to know that subtrees also contain dummy keys. The sequence has no actual keys but does contain the single dummy key d_{i-1} . Symmetrically, if we select k_j as the root, then k_j ’s right subtree contains the keys, $k_{j+1} \dots, k_j$; this right subtree contains no actual keys, but it does contain the dummy key d_j .

Step2: A recursive solution

- We are ready to define the value of an optimal solution recursively. We pick our subproblem domain as finding an OBST containing the keys $k_i, \dots, k_j$, where $i \geq 1$, $j \leq n$, and $j \geq i-1$. (It is when $j=i-1$ that there are no actual keys; we have just the dummy key d_{i-1} .)
- Let us define $e[i,j]$ as the expected cost of searching an OBST containing the keys $k_i, \dots, k_j$. Ultimately, we wish to compute $e[1,n]$.

- The easy case occurs when $j=i-1$. Then we have just the dummy key d_{i-1} . The expected search cost is $e[i,i-1] = q_{i-1}$.
- When $j \geq i$, we need to select a root k_r from among $k_i, \dots, k_j$ and then make an OBST with keys $k_i, \dots, k_{r-1}$ its left subtree and an OBST with keys $k_{r+1}, \dots, k_j$ its right subtree. By the time, what happens to the expected search cost of a subtree when it becomes a subtree of a node? The answer is that the depth of each node in the subtree increases by 1.

- By the second statement, the expected search cost of this subtree increases by the sum of all the probabilities in the subtree. For a subtree with keys $k_i, \dots, k_j$ let us denote this sum of probabilities as

$$w(i, j) = (i \sim j) \sum p_i + (i-1 \sim j) \sum q_i$$

Thus, if k_r is the root of an optimal subtree containing keys $k_i, \dots, k_j$, we have

$$E[i, j] = p_r + (e[i, r-1] + w(i, r-1)) + (e[r+1, j] + w(r+1, j))$$

Nothing that $w(i, j) = w(i, r-1) + p_r + w(r+1, j)$

- We rewrite $e[i,j]$ as
$$e[i,j] = e[i,r-1] + e[r+1,j] + w(i,j)$$

The recursive equation as above assumes that we know which node k_r to use as the root. We choose the root that gives the lowest expected search cost, giving us our final recursive formulation:

$E[i,j] =$

case1: if $i \leq j, i \leq r \leq j$

$$E[i,j] = \min\{e[i,r-1] + e[r+1,j] + w(i,j)\}$$

case2: if $j = i-1$; $E[i,j] = q_{i-1}$

- The $e[i,j]$ values give the expected search costs in OBST. To help us keep track of the structure of OBST, we define $root[i,j]$, for $1 \leq i \leq j \leq n$, to be the index r for which k_r is the root of an OBST containing keys $k_i, \dots, k_j$.

Step3: Computing the expected search cost of an OBST

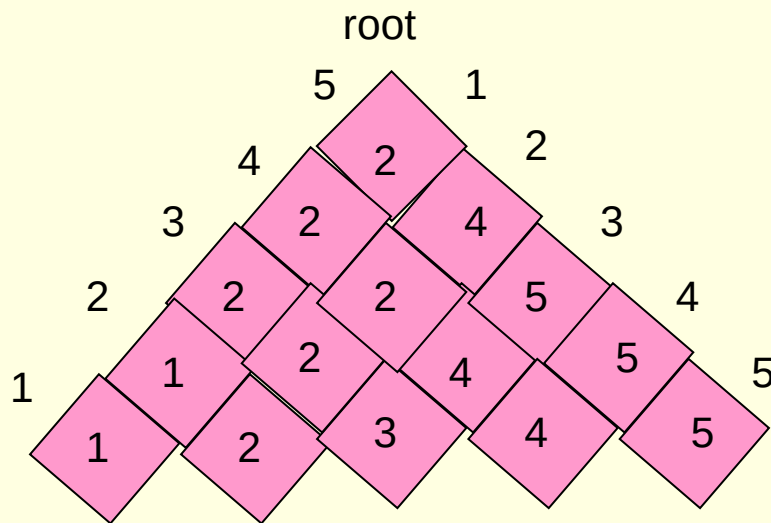
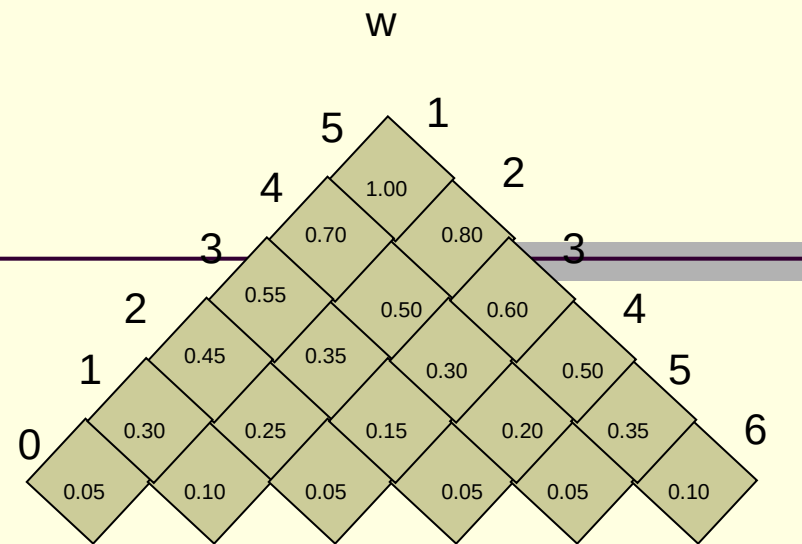
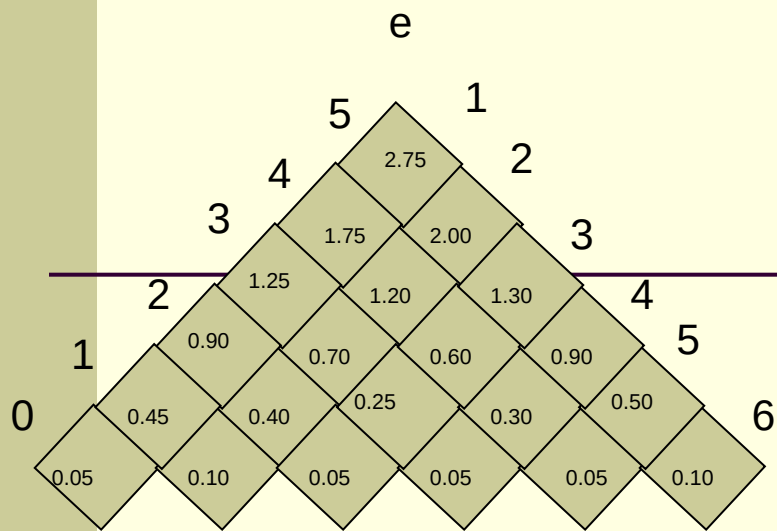
- We store the $e[i,j]$ values in a table $e[1..n+1, 0..n]$. The first index needs to run to $n+1$ rather than n because in order to have a subtree containing only the dummy key d_n , we will need to compute and store $e[n+1,n]$. The second index needs to start from 0 because in order to have a subtree containing only the dummy key d_0 , we will need to compute and store $e[1,0]$. We will use only the entries $e[i,j]$ for which $j \geq i-1$. We also use a table $root[i,j]$, for recording the root of the subtree containing keys $k_i, \dots, k_j$. This table uses only the entries for which $1 \leq i \leq j \leq n$.

- We will need one other table for efficiency.
Rather than compute the value of $w(i,j)$ from scratch every time we are computing $e[i,j]$ -----
we store these values in a table $w[1..n+1,0..n]$.
For the base case, we compute $w[i,i-1] = q_{i-1}$ for $1 \leq i \leq n$.
- For $j \geq 1$, we compute :
$$w[i,j] = w[i,j-1] + p_i + q_j$$

OPTIMAL—BST(p,q,n)

```

■ For i    1 to n+1
  ← do e[i,i-1]    qi-1
    do w[i,i-1]    qi-1
  For l    1 to n
  ← do for i    1 to n-l +1
    ← do j    i+l-1
      ← e[i,j]    ∞
      ← w[i,j]    w[i,j-1]+pj+qj
      ← For r    i to j
        ← do t    e[i,r-1]+e[r+1,j]+w[i,j]
          ← if t < e[i,j]
            then e[i,j]    t
            root[i,j]    r
  ←
Return e and root
  
```



The tables $e[i,j]$, $w[i,j]$, and $root[i,j]$ computed by Optimal-BST

Exercise

i	0	1	2	3	4	5	6	7
pi		0.04	0.06	0.08	0.02	0.10	0.12	0.14
qi	0.06	0.06	0.06	0.06	0.05	0.05	0.05	0.05